

SK Testing Platform Report

ABSTRACT

SK Testing Platform is a full-stack autonomous quality assurance and website analysis system developed to inspect a target website, observe real runtime behavior, generate structured test cases, and present actionable quality evidence through a centralized dashboard. The platform combines a React-based operator console with a Node.js, Express, Playwright, and MongoDB-powered analysis engine.

The system is designed to automate major parts of exploratory frontend validation. It recursively crawls the target website, discovers routes and interactive elements, executes controlled interactions, captures screenshots and HTML previews, and records runtime issues such as failed requests, JavaScript exceptions, accessibility risks, boundary-related failures, and API contract concerns. In addition to frontend inspection, the platform can correlate observed API usage with optional backend repository or upload-path input supplied by the operator.

The project follows a centralized architecture in which analysis runs are created, queued, processed by a worker layer, persisted to storage, and streamed back to the frontend in real time. The generated outputs include coverage summaries, page inventories, interaction evidence, grouped failures, API assertions, generated test cases, report summaries, and run history comparisons.

This project is intended to support QA engineers, developers, and reviewers by reducing the manual effort required to inspect application behavior and by packaging testing evidence into a form suitable for review, reporting, and future extension.

CONTENT TABLE OF CONTENTS

1. [Chapter 1: Introduction](#) 1.1 [Brief Overview of Work](#) 1.2 [Objective](#) 1.3 [Scope](#) 1.4 [Project Modules](#) 1.5 [Project Requirements](#)
2. [Chapter 2: System Analysis](#) 2.1 [Existing Need and Context](#) 2.2 [Project Feasibility Study](#) 2.3 [Project Timeline Summary](#) 2.4 [Detailed Module Description with Functionalities](#)
3. [Chapter 3: System Design](#) 3.1 [Architectural Design](#) 3.2 [Data Flow](#) 3.3 [Core Data Model](#) 3.4 [Page and Interaction Design](#) 3.5 [Activity Flow](#)
4. [Chapter 4: Software Tools](#) 4.1 [React and Vite](#) 4.2 [TypeScript](#) 4.3 [Express.js](#) 4.4 [Playwright](#) 4.5 [MongoDB](#) 4.6 [Tailwind CSS and Frontend UI Utilities](#)
5. [Chapter 5: Implementation and Testing](#) 5.1 [User Interface and Application Pages](#) 5.2 [Implemented Functional Testing Coverage](#)
6. [Chapter 6: Conclusion and Future Scope](#)
7. [References](#)

CHAPTER 1: INTRODUCTION

1.1 Brief Overview of Work

SK Testing Platform is built as an autonomous testing workspace for modern web applications. The main purpose of the project is to allow an operator to submit a target website and receive a structured analysis of

the website's navigational flow, interactive behavior, quality risks, backend interaction patterns, and generated test scenarios.

The application operates as a two-part system. The frontend provides a dense operator dashboard through which the user starts analysis runs, monitors live execution, reviews captured pages and findings, inspects generated test cases, and compares historical runs. The backend performs the actual analysis work by crawling the target website with Playwright, observing runtime behavior, storing analysis state, and producing final report data.

The platform has been implemented as a practical QA workflow tool rather than a static reporting prototype. It includes queued execution, run history, failure recovery, checkpoint-based authentication handling, screenshot evidence, and report generation, making it suitable for iterative website analysis.

1.2 Objective

The main objectives of SK Testing Platform are:

- to automate repetitive frontend exploration and interaction analysis;
- to generate structured QA evidence from a live website instead of relying only on manual inspection;
- to detect runtime issues such as request failures, JavaScript problems, accessibility concerns, visual anomalies, and boundary condition failures;
- to create categorized test cases from observed application behavior;
- to maintain a persistent record of runs, artifacts, logs, and findings for later review;
- to enable comparison between historical runs so that regressions, fixes, and persistent defects can be identified quickly.

1.3 Scope

The project is focused on automated and semi-autonomous web application testing. Its scope includes frontend crawling, interaction testing, quality observation, result persistence, and operator reporting. The current implementation supports:

- recursive page discovery within configured crawl limits;
- route-wise page inventory generation;
- interaction testing for buttons and other actionable elements;
- screenshot capture and preview generation;
- backend/API traffic observation;
- failure clustering and runtime finding generation;
- test-case generation across multiple QA categories;
- live monitoring of analysis progress through server-sent events;
- comparison of two runs for delta-based quality review.

The current scope does not represent a full production-grade enterprise test orchestration platform yet, but it already provides a strong operational foundation for autonomous website quality analysis.

1.4 Project Modules

The SK Testing Platform is divided into the following major modules.

1.4.1 Run Workspace Module

This module allows the operator to submit a target URL, specify backend context, configure analysis behavior, save target projects, retry failed runs, and respond to login checkpoints.

1.4.2 Frontend Crawling Module

This module discovers pages, extracts headings, buttons, links, inputs, forms, and interaction notes, and captures page previews and HTML snapshots.

1.4.3 Deep Analysis Module

This module performs accessibility checks, boundary probes, scenario execution, visual and runtime inspection, and API assertion generation.

1.4.4 Test Generation Module

This module converts collected frontend evidence into structured test cases such as functional, negative, boundary, edge, UX, integration, and performance cases.

1.4.5 Backend Correlation Module

This module identifies observed API endpoints, detects failed API assertions, and maps frontend behavior against optional backend ownership signals.

1.4.6 Reporting and Comparison Module

This module assembles report sections, a Mermaid flowchart, PDF-ready outline content, run history, and two-run comparison output.

1.5 Project Requirements

1.5.1 Hardware

The project is intended to run on a development machine capable of supporting:

- a modern browser runtime for Playwright-driven crawling;
- sufficient memory for concurrent crawl sessions and artifact capture;
- local or remote storage access for analysis persistence;
- normal frontend and backend development workloads.

1.5.2 Software

The currently implemented software stack includes:

- React 19;
- Vite 8;
- TypeScript 6;
- Express 5;
- Playwright 1.55;
- MongoDB through Mongoose;

- Tailwind CSS 4;
- Node.js runtime and npm-based package management.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing Need and Context

Website quality assurance is often fragmented across manual exploration, browser inspection, API inspection, and documentation review. In many cases, engineers and testers must repeatedly open pages, click through flows, record screenshots, verify API behavior, and convert those observations into test cases and reports. This process is time-consuming and inconsistent.

SK Testing Platform addresses this need by consolidating observation, interaction testing, evidence capture, and reporting into a single system. Instead of treating crawl data, runtime findings, screenshots, and generated tests as separate activities, the platform connects them into one analysis pipeline.

2.2 Project Feasibility Study

2.2.1 Technical Feasibility

The project is technically feasible because its core requirements are well supported by the chosen stack. Playwright provides browser automation and runtime inspection, Express supports the API layer, React supports the operator dashboard, and MongoDB provides persistence for runs and associated artifacts. The codebase already demonstrates successful integration of these components.

2.2.2 Economical Feasibility

The project uses broadly available open-source technologies and is economical for development and iterative expansion. It does not depend on expensive proprietary testing infrastructure for its current feature set. The main operational costs would arise from scaling storage, compute, and artifact retention as usage grows.

2.2.3 Operational Feasibility

The platform is operationally feasible because it exposes a workflow familiar to QA and engineering teams: submit a target, observe progress, review findings, inspect generated tests, and compare results over time. The interface is already organized into practical views that support this flow directly.

2.3 Project Timeline Summary

The present codebase reflects a staged implementation pattern:

1. frontend workspace and navigation setup;
2. backend API and health-route setup;
3. analysis queue and worker orchestration;
4. crawler and interactive element extraction;
5. deep analysis and test generation;
6. persistence, streaming, and report generation;
7. history and compare workflow integration.

2.4 Detailed Module Description with Functionalities

2.4.1 Run Workspace Module

Implemented functionalities:

- target URL validation;
- backend repository URL input;
- uploaded backend path input;
- saved-project persistence;
- pinned project support;
- retry of failed runs;
- fatal error display;
- login-choice handling;
- dedicated login session creation;
- live preview and elapsed-time tracking.

2.4.2 Frontend Crawling Module

Implemented functionalities:

- same-origin recursive crawl;
- route discovery;
- heading extraction;
- button, link, and input enumeration;
- form descriptor extraction;
- discovered-link tracking;
- interaction notes;
- robots.txt-aware crawling;
- HTML preview generation;
- preview image capture.

2.4.3 Deep Analysis Module

Implemented functionalities:

- accessibility inspection;
- visual behavior checks;
- boundary and limit probes;
- scenario pack execution;
- runtime warning generation;
- API assertion generation;
- response-shape inference.

2.4.4 Test Generation Module

Implemented functionalities:

- functional test generation;
- negative test generation;
- boundary test generation;

- edge test generation;
- UX test generation;
- integration test generation;
- performance test generation;
- issue-aware suggestion attachment.

2.4.5 Backend Correlation Module

Implemented functionalities:

- API endpoint extraction from observed requests;
- matched endpoint summary;
- mismatched endpoint summary;
- failed assertion tracking;
- ownership signal capture from repo URL or upload path;
- frontend/backend contract observation text generation.

2.4.6 Reporting and Comparison Module

Implemented functionalities:

- overview report section;
- issues report section;
- performance report section;
- Mermaid flowchart output;
- PDF-outline output;
- historical run listing;
- two-run diff classification into new, fixed, and persistent findings.

CHAPTER 3: SYSTEM DESIGN

3.1 Architectural Design

The platform follows a layered architecture.

1. Presentation Layer The React frontend provides the operator console and displays live and completed analysis data.
2. API Layer The Express backend exposes routes for run creation, run listing, run retrieval, retry, checkpoint continuation, and SSE streaming.
3. Orchestration Layer The analysis service manages queued runs, worker leasing, progress buffering, and final result assembly.
4. Analysis Layer The Playwright crawler and deep-analysis helpers inspect the target website and generate raw evidence.
5. Persistence and Reporting Layer MongoDB-backed storage and report builders preserve the analysis state and package the output.

3.2 Data Flow

The overall flow of data in the system is:

1. Operator submits website details.
2. Frontend sends analysis request to backend.
3. Backend creates queued run.
4. Worker claims run and starts crawling.
5. Pages, interactions, logs, artifacts, and findings are collected.
6. Partial updates are streamed back to the frontend.
7. Test generation and backend validation are executed.
8. Final report data is assembled and stored.
9. Completed results become available in all relevant frontend views.

3.3 Core Data Model

The main stored and transmitted entities in the project are:

- `AnalysisRun`
- `AnalysisPage`
- `AnalysisInteraction`
- `AnalysisArtifact`
- `AnalysisLog`
- `FrontendAnalysis`
- `GeneratedTestCase`
- `BackendValidationResult`
- `AnalysisReport`

Important data fields represented by the system include:

- run identifiers and run lineage;
- run status and progress information;
- page URL, route, headings, buttons, links, and forms;
- interaction results with pass or fail status;
- runtime findings with severity and evidence;
- screenshots and preview URLs;
- test cases and expected results;
- backend validation observations.

3.4 Page and Interaction Design

The frontend is intentionally divided into focused operational pages.

- `Overview` presents summary metrics, top findings, and suggestions.
- `Run Lab` handles submission, live progress, login checkpoints, and saved projects.
- `Pages` displays route-wise crawl output.
- `Findings` shows runtime issues with filter controls.
- `Tests` presents generated test cases and diff evidence.
- `Report` organizes flow, findings, backend observations, and outline data.

- **History** stores and reopens prior runs.
- **Compare** highlights new, fixed, and persistent findings.

3.5 Activity Flow

The activity flow of the system can be described as:

1. User enters target details.
2. System validates input.
3. Analysis run is created.
4. Worker begins crawl and interaction testing.
5. System stores pages, interactions, logs, and artifacts.
6. If login is needed, checkpoint is raised.
7. Analysis resumes or branches into a dedicated login session.
8. Deep analysis and test generation are completed.
9. Report and result package are finalized.
10. User reviews output and may compare it with previous runs.

CHAPTER 4: SOFTWARE TOOLS

4.1 React and Vite

React is used to build the operator interface and handle view-level rendering, stateful interaction, and dynamic updates. Vite is used as the frontend development and build tool, providing fast local development and optimized bundling.

4.2 TypeScript

TypeScript is used across both frontend and backend to define structured types for runs, pages, interactions, findings, test cases, reports, and API contracts. It improves consistency and reduces integration ambiguity between modules.

4.3 Express.js

Express provides the backend HTTP layer. It is responsible for API routing, JSON parsing, static artifact serving, health checking, and centralized error handling.

4.4 Playwright

Playwright is the main browser automation and runtime observation tool used by the project. It supports page navigation, element interaction, screenshot capture, DOM inspection, network observation, and login-aware browser sessions.

4.5 MongoDB

MongoDB, accessed through Mongoose, is used for persisting analysis runs and related data. It supports the storage of logs, pages, artifacts, progress snapshots, interactions, and completed analysis results.

4.6 Tailwind CSS and Frontend UI Utilities

Tailwind CSS is used for frontend styling and helps implement the dashboard's dense visual layout. Additional frontend utility components handle navigation, filtering, modals, empty states, mobile actions, and status displays.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 User Interface and Application Pages

The implemented user interface includes the following major application pages and operational views.

Overview Page

- shows overall coverage;
- shows passed and failed interaction summary;
- shows issue count;
- shows top findings and suggestions.

Run Lab Page

- starts new analysis runs;
- accepts target URL and optional backend signals;
- shows current run stage, timing, logs, and previews;
- handles retry and login continuation.

Pages Page

- displays route tree;
- allows page search;
- shows headings, buttons, links, forms, and notes for selected pages.

Findings Page

- displays runtime findings with severity and type;
- supports route, severity, and issue-type filtering;
- shows screenshot-backed issue cards.

Tests Page

- displays generated test cases;
- prioritizes failed cases;
- shows steps, expected result, issue summary, suggestions, and diff evidence.

Report Page

- displays filtered flow sequence;
- displays backend validation observations;
- shows report-oriented findings summary;
- displays PDF-outline content.

History Page

- stores all previous runs;
- supports search and filtering;
- allows run reopening and comparison selection.

Compare Page

- compares exactly two runs;
- shows coverage and failure delta;
- classifies findings as new, fixed, or persistent.

5.2 Implemented Functional Testing Coverage

The project itself is designed to generate and represent test evidence. Based on the implemented code, the following functional testing coverage is built into the platform.

Run Submission and Validation

- valid website URL acceptance;
- invalid URL rejection;
- backend input attachment;
- saved-project restore and reuse;
- retry trigger behavior.

Crawl and Discovery Verification

- page discovery within limits;
- route-path extraction;
- heading, button, link, and form capture;
- HTML preview and preview-image generation.

Interaction Verification

- button interaction execution;
- DOM-change tracking;
- URL-change tracking;
- request-trigger detection;
- screenshot generation before and after interactions;
- failure evidence capture.

Authentication and Checkpoint Verification

- login-surface detection;
- checkpoint wait state;
- continue without login flow;
- continue through dedicated login session flow.

Reporting Verification

- report-section creation;
- flowchart generation;
- outline generation;
- backend validation observation generation.

History and Comparison Verification

- run listing;
- single-run reopening;
- dual-run comparison;
- new, fixed, and persistent finding classification.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

Conclusion

SK Testing Platform is a practical autonomous QA system that integrates crawling, runtime inspection, artifact capture, test generation, backend correlation, and reporting into one workflow. The current implementation already goes beyond a simple dashboard or crawler by supporting queued execution, live streaming, authentication checkpoints, historical comparisons, and structured evidence output.

The system is especially valuable because it reduces manual observation effort while preserving actionable detail. Instead of requiring testers to manually record pages, interactions, screenshots, and findings, the platform collects that evidence directly and organizes it into pages, findings, tests, and reports.

Future Scope

The following areas provide strong future expansion opportunities:

- executable replay of generated test cases as a formal regression suite;
- deeper authenticated crawling with reusable session profiles;
- broader scenario packs for domain-specific applications;
- PDF export of the final report in polished institutional format;
- richer analytics for flaky behavior and stability trends;
- multi-user collaboration and role-based review workflows;
- CI/CD integration for automated scheduled analysis;
- stronger production-grade artifact management and retention policies.

REFERENCES

Technologies and Frameworks

- React documentation
- Vite documentation
- TypeScript documentation
- Express.js documentation
- Playwright documentation
- MongoDB and Mongoose documentation
- Tailwind CSS documentation

Project Source References

- frontend application components and runtime configuration
- backend API routes and orchestration modules
- crawler, deep analysis, reporting, and persistence modules
- shared type definitions used by the frontend and backend